

One Person Lab

One Person Lab 白皮书

从一人实验室到可运行的智能体组织系统

OPL 的核心承诺，是把高价值知识工作从一次性对话，升级成有阶段、有证据、有 owner、有工作空间、有交付物的智能体运行系统。

OPL Framework / One Person Lab App / Foundry Agents

2026-06-07

Public whitepaper

目录

1 定位摘要 2

2 一人实验室需要新的操作系统 2

3 OPL 的三层产品模型 2

3.1 OPL Framework 2

3.2 One Person Lab App 3

3.3 Foundry Agents 3

4 设计哲学：让 AI 做专家工作，让系统守住边界 3

4.1 AI-first 3

4.2 Executor-first 3

4.3 Stage-led 3

4.4 Contract-light 4

4.5 Owner boundary 4

4.6 Progress-first 4

5 九个品牌模块：从上到下的设计骨架 4

5.1 OPL Charter 4

5.2 OPL Atlas 4

5.3 OPL Workspace 4

5.4 OPL Stagecraft 5

5.5 OPL Runway 5

5.6 OPL Vault 5

5.7 OPL Console 5

5.8 OPL Foundry Lab 5

5.9 OPL Connect 5

6 Foundry Agents：同一骨架，不同领域 6

6.1 MedAutoScience 6

6.2 MedAutoGrant 6

6.3 RedCube AI 6

6.4 OPL Meta Agent 6

6.5 未来 Foundries 6

7 为什么 OPL 能稳定运行 7

8 用户会如何感知 OPL 7

9 这份白皮书如何维护 7

10 参考与编制来源 8

适用对象：希望理解 OPL 品牌、产品定位、设计逻辑和 Foundry Agents 运行方式的用户、合作者、早期采用者和技术决策者。

核心判断：OPL 的核心承诺，是把高价值知识工作从一次性对话，升级成有阶段、有证据、有 owner、有工作空间、有交付物的智能体运行系统。

1 定位摘要

- OPL 不是一个聊天入口集合，而是面向正式知识工作的智能体运行框架。
- One Person Lab App 是普通用户的工作台，负责把 framework 和 Foundry Agents 变成可见、可操作、可检查的产品体验。
- Foundry Agents 是运行在 OPL 上的领域智能体，例如 Research、Grant、Presentation 和未来的 IP、Award、Thesis、Review。
- OPL 的设计从顶层目标反推到底层合同：用户要的是可靠交付，系统就必须提供阶段、权限、证据、恢复和交接。

2 一人实验室需要新的操作系统

过去，一个人可以通过笔记、脚本、文献管理器和通用 AI 对话工具完成很多工作。但当任务进入论文、基金、专利、汇报、审稿、答辩这样的正式场景时，真正的瓶颈不再是单次生成能力，而是持续推进能力。

正式知识工作有几个共同特征：材料多、周期长、阶段清楚、质量要求高、交付物必须能被复核，并且每一步都需要知道谁拥有判断权。一次对话可以帮人写一段文字，却很难天然承担数据整理、证据归档、质量门、修订历史、下一责任方和最终交付这些长期责任。

One Person Lab 的出发点，是让一个人也能像一个小型实验室一样组织工作。用户不需要亲自管理每个底层执行细节，但需要始终知道当前任务在哪个阶段、产出了什么、缺什么证据、谁应该继续行动，以及最终交付物在哪里。

OPL 因此把 AI 能力放进一个从上到下设计的运行系统里。系统为 AI executor 提供目标、上下文、工具、权限、质量门和交接边界；AI executor 负责开放式推理、创作、评审、诊断和修订；领域智能体负责自己的专业 truth 和质量裁决；App 把这些状态转成用户能看懂的工作台。

- 用户看到的是任务进展和交付物，不是零散日志。
- Agent 接到的是阶段目标和可用能力，不是被硬编码的机械脚本。
- 系统保存的是证据、回执、阻塞和恢复点，不是无法复盘的聊天片段。

3 OPL 的三层产品模型

OPL 对外是一个品牌，对内是清楚分层的系统：Framework、App、Foundry Agents。

默认运行链路

```
Human / App / CLI
  -> Codex-default executor
      -> explicit OPL activation
          -> provider-backed stage runtime
              -> selected Foundry Agent
                  -> domain-owned truth / quality gate / deliverables
```

3.1 OPL Framework

运行框架

Framework 持有 Codex-default session runtime、显式 activation layer、stage control plane、typed queue、provider-backed runtime、shared contracts、恢复和审计 surface。它负责让任务可以被启动、恢复、投影和收口。

- 默认第一公民 executor 是 Codex CLI。
- Temporal-backed provider 是 production online runtime 的必需 substrate。

- Framework 不持有 domain truth、quality verdict 或 artifact authority。

3.2 One Person Lab App

用户工作台

App 是普通用户使用 OPL 的产品入口。它消费 Framework 和 domain-owned projection，把任务、阶段、阻塞、交付物、source refs 和 next action 呈现为可操作界面。

- App 固定 Codex CLI concrete executor 的普通用户路径。
- App 负责 GUI product truth、release gate、用户教程和 active shell validation。
- App 不成为第二 runtime，也不替 domain agent 声明质量完成。

3.3 Foundry Agents

领域智能体

Foundry Agents 是 MAS、MAG、RCA、OMA 以及未来 Patent、Award、Thesis、Review 等领域智能体。它们以统一 OPL 结构运行，但保留自己的领域判断、质量 gate、artifact authority 和 direct skill path。

- 同一生命周期：材料进入、stage 执行、质量门、owner receipt、交付物 handoff。
- 不同领域保留不同输入、输出、知识、rubric 和 authority function。
- OPL 只投影 opaque refs，不读取或改写领域 body。

4 设计哲学：让 AI 做专家工作，让系统守住边界

OPL 的系统感来自一组明确取舍。它不是用更复杂的规则替代 AI，而是把 AI 的开放式能力放进可审计、可恢复、可交接的边界内。

4.1 AI-first

能力来自更强执行者

复杂知识工作应该交给 AI executor 做开放式判断、写作、评审和修订。代码不把这些专家行为写死，只提供目标、上下文、权限、工具和质量边界。

- 模型升级、prompt 改进、skill 改进会直接提升系统能力。
- 合同只定义下限，不替代专家判断。

4.2 Executor-first

stage 内最小执行单位是 Agent executor

OPL 不是把任务拆成大量细碎函数，而是让一个被选中的 executor 在一个 stage 内完成接近人类专家的真实工作包。

- 默认 executor 是 Codex CLI。
- 非默认 executor 只通过显式 adapter 接入，并必须有 receipt、audit 和 fail-closed 语义。

4.3 Stage-led

大型任务按阶段推进

论文、基金、视觉交付和答辩材料都有自然阶段。OPL 把 stage 作为可观察、可恢复、可审计的工作单元，而不是把知识工作压成固定脚本。

- 每个 stage 有目标、输入、输出、知识、工具边界和质量门。
- stage closeout 必须回到 owner receipt、typed blocker、human gate 或 route-back。

4.4 Contract-light

合同只守安全和可恢复下限

合同负责 owner、权限、safe action、receipt、blocker、audit、recovery 和 projection。它不声明论文质量、基金可中、视觉作品通过，也不把 readiness 当成 domain verdict。

- 机械检查只能定位缺口。
- 质量裁决必须来自独立 reviewer、domain quality gate 或 owner receipt。

4.5 Owner boundary

每个判断都有责任方

Framework、App 和 Foundry Agents 各有自己的 authority。OPL 管通用运行外围，domain agent 管领域 truth，App 管用户界面和 release evidence。

- 不制造第二真相源。
- 不让 UI 或 runtime 反向定义领域质量。

4.6 Progress-first

用户需要看见下一步

OPL 默认把 owner delta 放在最前面：当前等待谁、需要什么交付 delta、缺什么 receipt 或 blocker。用户看到的是可推进状态，而不是被底层计数淹没。

- 进度来自产物、证据、决策和交接。
- 诊断细节只在 drilldown 中展开。

5 九个品牌模块：从上到下的设计骨架

九个模块不是营销名称，而是 OPL Framework 内部能力的 bounded context。它们让用户感知到 OPL 不是一堆功能拼装，而是一套从宪章、目录、工作空间、阶段设计、运行、证据、控制台、智能体工坊到外部连接的完整结构。

5.1 OPL Charter

语言和边界

Charter 固定 OPL 的顶层定位、命名、产品层级、ADR/RFC 和品牌组合治理。它回答什么叫 OPL、什么属于 Framework、什么属于 App、什么属于 Foundry Agent。

- 让整个系统拥有统一语言。
- 防止旧路线和新路线混在同一叙事里。

5.2 OPL Atlas

可发现目录

Atlas 管理 agents、capabilities、surfaces、owners、dependencies 和 lifecycle catalog。它让用户和系统知道有哪些智能体、能力、入口和 owner。

- 负责 discoverability，不执行动作。
- 把 domain descriptor、module registry 和 public surface 放进统一目录。

5.3 OPL Workspace

用户和 Agent 共同检查的项目空间

Workspace 把材料、共享资源、stage outputs、handoff 和文件生命周期组织成可检查结构。用户能在项目空间里看到当前产物和交付物，而不必钻进 runtime backing directory。

- 默认模型是 Workspace Group -> Project Unit -> Stage Artifact Unit。
- 真实完成必须绑定 manifest、receipt 和 current pointer。

5.4 OPL Stagecraft

stage 内认知计算设计

Stagecraft 负责 stage 设计、prompt、skills、tool affordance、knowledge、rubric 和 independent quality gate。它保留 AI 的开放式专家空间，同时让输入、输出和边界可审计。

- 工具目录是 affordance catalog，不是硬编码流程。
- 复杂质量判断必须是独立 stage 或 quality gate。

5.5 OPL Runway

可恢复的长跑执行

Runway 负责 durable execution、typed queue、attempt、lease、retry/dead-letter、wakeup 和 human gate。它让一个长期任务不会因为进程退出或上下文切换而失去运行线索。

- 承接 provider-backed stage runtime。
- 不创建 domain verdict，不替代 executor 的专家工作。

5.6 OPL Vault

证据、回执和 lineage

Vault 保存 refs-only evidence、receipt refs、typed blocker refs、artifact lineage、restore proof 和 provenance。它记录任务为什么可以继续、为什么被阻塞、如何恢复。

- 保存 refs，不保存 domain memory 或 artifact body。
- 让系统可审计，但不篡改领域 truth。

5.7 OPL Console

用户和 operator 的可见控制台

Console 把 readiness、current owner、next action、阻塞、产物和 drilldown 投影到 App/operator 工作台。它让用户知道任务在哪里，而不是让用户读底层 ledger。

- 默认 owner-delta-first。
- 只消费 projection，不成为第二 runtime。

5.8 OPL Foundry Lab

智能体创建与机制改进

Foundry Lab 支撑新 agent 创建、测试接管、mechanism improvement、canary、promotion 和 rollback。它把运行证据转成可执行改进任务。

- 用于改进 agent 机制。
- 不接管 MAS、MAG、RCA 或未来 agent 的 domain authority。

5.9 OPL Connect

外部调用和分发连接

Connect 把同一合同派生到 CLI、MCP、OpenAI/AI SDK tools、Skill/plugin、module install、release/install 分发和 drift matrix。

- 让能力可以被安装、同步、发现和调用。
- 不重新解释 domain 语义。

6 Foundry Agents: 同一骨架，不同领域

OPL 的品牌感很大一部分来自 Foundry Agents。每个 agent 都面向一个高价值工作流，拥有自己的领域知识、质量门和交付权威，同时共享 OPL 的 stage-led 运行骨架。

6.1 MedAutoScience

Research Foundry

面向医学和科研论文工作流，从数据、文献、研究问题、分析、证据包到稿件和投稿包。它持有 research truth、publication quality gate、artifact authority 和 owner receipt。

- 用户感知：把研究项目持续推进到可审阅论文和投稿材料。
- OPL 只承载 stage attempt、queue、receipt projection 和 workspace/runtime 支撑。

6.2 MedAutoGrant

Grant Foundry

面向基金方向判断、申请书写作、作者侧模拟评审和修订。它复用研究证据和记忆，但保持 grant truth 与 fundability review 边界。

- 用户感知：从想法到申请书草稿、模拟评审包和修订计划。
- 质量判断回到 MAG 自己的 gate 和 owner receipt。

6.3 RedCube AI

Presentation Foundry

面向视觉交付、PPT、报告、storyboard、render、review 和 export。它持有 visual truth、review/export verdict 和 artifact authority。

- 用户感知：把材料转成可展示的视觉交付物。
- OPL 负责 stage folder、manifest、receipt refs 和投影。

6.4 OPL Meta Agent

Foundry Lab managed module

面向 agent 创建、测试接管、机制改进和 target-agent work order。它帮助 OPL family 变得更会生产 agent，但不持有任何领域 agent 的最终 truth。

- 用户感知：OPL 能持续变强、持续生成和修复自己的 agents。
- 边界：只生成改进 work order 或 typed blocker，不替 domain owner 签收。

6.5 未来 Foundries

IP / Award / Thesis / Review

IP、Award、Thesis、Review 等工作流会沿用同一骨架：材料进入、domain pack 解释、stage-led 执行、独立质量门、owner receipt 或 typed blocker、交付物 handoff、OPL refs-only projection。

- 每个新 agent 都必须有自己的 domain truth 和质量边界。

- 共享 OPL runtime，不复制私有通用 runtime。

7 为什么 OPL 能稳定运行

OPL 的稳定感不是来自把所有事情写成固定流程，而是来自把开放式智能工作放进一组可靠边界里。

第一，任务以 stage 为单位推进。一个 stage 足够大，可以让 AI executor 完成真实工作包；同时也足够清楚，可以定义输入、输出、权限、quality gate 和 closeout 形态。

第二，owner boundary 清楚。Framework 只持有 framework truth；App 只持有产品和展示 truth；Foundry Agent 持有 domain truth、quality verdict 和 artifact authority。这样每个判断都有归属，系统不会因为投影可见就误报完成。

第三，证据是 refs-only 的。OPL 不复制领域正文，不用 UI 或 ledger 伪造质量裁决。它记录 receipt、typed blocker、lineage、current pointer 和 restore proof，让任务可以被审计和恢复。

第四，普通用户默认看到 next action。OPL 不把用户困在底层事件、trace、计数和 debugging lane 里，而是先回答当前等待谁、缺什么、下一步能做什么。

第五，系统从单一来源派生多个入口。CLI、App、Skill、MCP、AI SDK tool、plugin 和 release/install 分发都围绕同一 contract surface 生成，避免每个入口各说各话。

- 这就是 OPL 的核心体验：AI 有足够自由度完成专家工作，系统有足够纪律保证工作可见、可复核、可恢复、可交付。
- 用户不需要相信一段聊天记录。用户可以相信阶段、产物、回执、owner 和工作空间。

8 用户会如何感知 OPL

好的智能体系统不应该让用户感到自己在调试工具。它应该让用户觉得自己在管理一个可以持续推进工作的团队。

在 App 中，用户选择的是任务和 Foundry Agent，而不是 backend、adapter 或 executor 细节。普通用户路径固定为 Codex CLI executor，复杂 runtime 和 provider 状态折叠到 readiness、current owner、next action 和 drilldown。

在工作空间中，用户看到材料、stage outputs、交付物和当前指针。运行状态可以被解释，但不会取代项目文件本身。真正的交付物仍落在用户可以检查、复制、分享和归档的位置。

在长期任务中，用户看到每个阶段的进度、阻塞和交接。任务可能需要人类批准、补材料、接受 route-back 或等待 domain owner receipt，但这些状态都会显式出现。

- OPL 把复杂性收进系统，把可行动信息呈现给用户。
- OPL 把 AI 的创造力留在 stage 内，把责任和证据留在系统边界上。
- OPL 把一次性助手升级成可以持续运行的实验室伙伴。

9 这份白皮书如何维护

白皮书本身也遵循 OPL 的维护哲学：单一内容源、可复现生成、生成后验证。

内容源是 docs/public/whitepaper/opl-whitepaper.source.json。Markdown、PDF 和 verification JSON 都从这份 JSON 派生。这样做可以避免同一份白皮书在网页、PDF 和仓库文本中出现多套不一致的正文。

PDF 使用 Pandoc + XeLaTeX 生成，字体采用本机可用的 Noto Sans CJK SC。生成脚本会渲染 PDF 页面、抽取文本、检查关键内容、记录页数和工具链路径，方便后续维护者确认产物仍可分享。

外部白皮书风格参考了经典项目的做法：白皮书应先建立定位和信念，再解释系统模型；同时应保留当前性边界，不把历史或愿景写成未经验证的 production-ready 声明。

- 更新正文：改 JSON。
- 重新生成：运行 `npm run docs:whitepaper`。
- 验证分享版：检查 PDF 渲染页和 verification JSON。

10 参考与编制来源

- [Ethereum Whitepaper](#): 参考其白皮书页面对愿景、系统模型和当前性说明的组织方式。
- [Bitcoin whitepaper PDF](#): 参考其短而正式的技术宣言式结构。
- [Pandoc User Guide](#): 本仓采用 Pandoc + XeLaTeX 生成 PDF。
- [Typst PDF documentation](#): 作为后续可能迁移到更现代 PDF 工具链的参考。